Кейс №5: Сервис записи на услуги

Форум «Умная индустрия» • Номинация: Программирование (Уровень: Продвинутый, 1.5x)

Полнофункциональный full-stack веб-сервис онлайн-записи клиентов на услуги

• Backend: Kotlin 2.0.21, Java 21 LTS (Virtual Threads), Spring Boot 3.3.5, Spring Security, JPA, Flyway.

• СУБД: PostgreSQL 16 с аппаратной защитой от двойного бронирования (GiST Exclusion Constraint).

• Frontend: React 18, TypeScript 5.6 (strict), Vite 6, TanStack Query v5, Tailwind CSS.

• Комплект поставки Кейса №5:

1) Развёрнутый прототип v1.0.0 + Docker Compose и автозагрузка демо-данных при старте;

2) API-документация (docs/api/openapi.yaml) и схема БД (docs/architecture/DB-SCHEMA.md);

3) Тестовые учётные записи (client@booking.local, admin@booking.local) и 33 автотеста;

4) Паспорт экспертного решения на 81–100 баллов: расчёт ROI (2021%), матрица рисков и паттерны внедрения.

1. Заказчик, контекст и решаемая проблема бизнеса

Универсальная структура кейса • Раздел 1

Целевой заказчик:

• Предприятия сферы услуг (барбершопы, салоны красоты, автосервисы, медицинские и студийные центры) со штатом от 3 до 25 специалистов и потоком 25–150 записей в сутки.

Ключевые проблемы при ручной записи через телефон и мессенджеры:

• Двойное бронирование (овербукинг): накладки при одновременной записи двух клиентов к одному мастеру на одно и то же время — репутационные потери и отток клиентов.

• Потеря до 22% обращений в нерабочие часы (20:00–10:00), когда администратор не отвечает.

• До 60% рабочего времени администратора тратится на однотипную переписку по выбору слотов.

Цель проекта:

• Автоматизировать выбор услуги, свободного времени и управление записями с математической гарантией отсутствия двойного бронирования на уровне ядра базы данных.

2. Исходные материалы и оговорка о синтетических данных

Универсальная структура кейса • Раздел 2 (Обязательная оговорка)

Автоматическая инициализация демо-данных при первом запуске (DemoDataBootstrapRunner.kt):

• 5 услуг длительностью 30 и 60 минут (от 900 до 2 500 руб.), 3 специалиста (Алексей Громов, Дмитрий Волков, Елена Смирнова), 7-дневное расписание смен (10:00–14:00 и 15:00–20:00, перерыв 14:00–15:00).

ВАЖНОЕ ПРЕДУПРЕЖДЕНИЕ (согласно Положению о форуме «Умная индустрия»):

1. Источник или метод генерации: демонстрационный каталог, расписание и тестовые записи создаются детерминированным сидером DemoDataBootstrapRunner по синтетическому шаблону предприятия сферы услуг.

2. Обоснование репрезентативности: данные достоверно моделируют комбинирование 30- и 60-минутных услуг, матрицу компетенций мастеров (M:N), обеденные перерывы и конкуренцию за слоты.

3. Этическая и юридическая оговорка: персональные данные реальных граждан не используются (152-ФЗ). Все ФИО вымышлены, email используют резервный домен @booking.local, а телефоны — диапазон +7900000000X.

3. Архитектура решения и ограничения проекта

Универсальная структура кейса • Разделы 3 и 4

Модульная архитектура серверного ядра (Kotlin 2.0 / Spring Boot 3.3.5):

• Изолированные доменные пакеты: identity (аутентификация), catalog (услуги и мастера), schedule (графики и расчёт слотов), booking (жизненный цикл записи и модерация), shared (конфигурация и ошибки).

Безопасность и управление сессиями:

• Сессионная аутентификация через HttpOnly, SameSite=Lax cookie (JSESSIONID), пароли хешируются BCrypt (cost=12).

Единый часовой пояс бизнеса и точность времени (ADR-007):

• В БД все временные метки хранятся строго в UTC (TIMESTAMPTZ), а клиентское приложение принудительно форматирует слоты по таймзоне заведения Europe/Moscow вне зависимости от настроек устройства клиента.

Финансовая целостность (ADR-009 Price Snapshot):

• В таблицу bookings записываются неизменяемые снимки service_name_snapshot и service_price_snapshot.

4. Защита от двойного бронирования (Критерий приёмки №1)

Критерий приёмки: «Нельзя создать две записи на один ресурс и время»

Почему обычные проверки в коде (SELECT COUNT) не работают под нагрузкой:

• При параллельных HTTP-запросах возникает состояние гонки (TOCTOU): оба потока видят, что слот свободен, и вставляют две конфликтующие записи.

Наше решение — аппаратный констрейнт исключения PostgreSQL GiST (V1__init_schema.sql):

```
ALTER TABLE bookings ADD CONSTRAINT no_overlapping_active_bookings
EXCLUDE USING gist (master_id WITH =, tstzrange(start_time, end_time, '[)') WITH &&)
WHERE (status IN ('PENDING', 'CONFIRMED'));
```

Ключевые преимущества и доказательство:

• Полуоткрытый интервал [start_time, end_time) позволяет стыковать записи вплотную (10:00–11:00 и 11:00–12:00).

• При отмене записи (CANCELLED / REJECTED) слот мгновенно освобождается без удаления истории из БД.

• Стресс-тест OverbookingConcurrencyIntegrationTest (20 потоков через CyclicBarrier): ровно 1 запись получает 201 Created, остальные 19 — 409 Conflict (SLOT_ALREADY_BOOKED).

5. Личный кабинет и защита личных данных (Критерий приёмки №2)

Критерий приёмки: «Доступ к личным данным ограничен авторизацией»

Интерактивный 4-шаговый сценарий онлайн-записи (BookingFlow):

• Шаг 1: Выбор услуги (30 или 60 мин) -> Шаг 2: Выбор специалиста -> Шаг 3: Выбор даты (горизонт 30 дней) -> Шаг 4: Выбор свободного слота и подтверждение записи.

Личный кабинет клиента (ClientCabinetPage):

• Просмотр предстоящих («Активные») и прошлых («История») визитов со статусами и ценами.

• Отмена предстоящей записи в 1 клик (до начала визита T_start > now) с мгновенным освобождением слота.

Изоляция персональных данных и защита от IDOR:

• Публичный эндпоинт GET /api/v1/availability возвращает только свободные интервалы без ФИО и телефонов клиентов.

• Эндпоинты /api/v1/bookings/** требуют авторизации; попытка отменить чужую запись по UUID блокируется кодом 403 ACCESS_DENIED.

6. Админ-панель и синхронизация расписания (Критерий приёмки №3)

Критерий приёмки: «Изменения расписания корректно отражаются в клиентском интерфейсе»

Административная панель (защищена ролью ROLE_ADMIN) включает 4 вкладки:

1. Модерация записей (BookingsModerationTab): фильтрация по дате, мастеру и статусу; переходы статусов PENDING -> CONFIRMED / REJECTED, CONFIRMED -> COMPLETED / CANCELLED.

2. Управление расписанием (ScheduleTab): настройка рабочих интервалов и обеденных перерывов мастера по дням недели (Пн–Вс) с валидацией пересечений.

3. Управление услугами (ServicesTab): создание и редактирование услуг, защита от деактивации при наличии будущих записей (409 ACTIVE_BOOKINGS_EXIST).

4. Управление мастерами (MastersTab): привязка оказываемых услуг (M:N) и управление активностью.

Мгновенное отражение на витрине клиента:

• AvailabilityCalculatorService вычисляет слоты на лету из актуального расписания за вычетом активных броней.

7. Оптимизация, быстродействие и результаты тестирования

Критерии экспертного решения (81–100 баллов) • Техническое совершенство

Оптимизация производительности:

• Устранение проблемы N+1 запросов в JPA через @EntityGraph(attributePaths = ["client", "master", "service"]) — загрузка списков записей за 1 SQL-запрос с LEFT JOIN FETCH.

• Включены виртуальные потоки Java 21 (spring.threads.virtual.enabled=true) для высокой конкурентности.

• Время расчёта свободных слотов GET /api/v1/availability составляет < 25 мс (при требовании NFR p95 <= 300 мс).

• При получении 409 SLOT_ALREADY_BOOKED TanStack Query автоматически инвалидирует кэш и перерисовывает сетку слотов без перезагрузки страницы.

Полное покрытие автотестами (33 теста — 100% PASS):

• Backend (16 тестовых классов JUnit 5 + MockK + Testcontainers PostgreSQL 16): доменная машина состояний, миграции Flyway, RBAC/Security, расчёт слотов и 20-поточный стресс-тест овербукинга.

• Frontend (17 тестов Vitest + React Testing Library): вход/регистрация, демо-аккаунты, запись, перехват 409, ЛК и админ-панель.

8. Экономический эффект (ROI) и матрица рисков

Критерии экспертного решения (81–100 баллов) • Бизнес-эффект

Расчёт экономического эффекта (салон на 5 мастеров, 750 визитов/мес, чек 1 800 руб., оборот 1,35 млн руб./мес):

• Ликвидация потерь от овербукинга (2,5% накладок): +34 200 руб./мес сохранённой выручки.

• Круглосуточная запись 24/7 в вечерние и ночные часы (+30 визитов/мес): +54 000 руб./мес.

• Мгновенное возвращение отменённых слотов в продажу: +18 000 руб./мес.

• Сокращение рутины администратора на 44 часа в месяц: +26 400 руб./мес.

Итоговые показатели ROI:

• Чистый ежемесячный эффект: 132 600 руб./мес (1 591 200 руб./год).

• Затраты: внедрение 45 000 руб. (CAPEX) + облачный VPS 2 500 руб./мес (OPEX).

• Срок окупаемости (Payback Period): 11 дней. Годовой ROI = 2021%.

9. Паттерны внедрения и сценарий демонстрации для жюри

Готовность к внедрению и быстрая проверка прототипа

3 готовых паттерна внедрения:

1. Single-Node Docker Compose: развёртывание за 1 минуту (docker compose up -d + ./gradlew bootRun).

2. Headless API / Виджет: подключение к любому сайту или Telegram Mini App по контракту docs/api/openapi.yaml.

3. Multi-Branch Ready: модульная архитектура готова к масштабированию на сеть филиалов.

Тестовые учётные записи (в форме входа есть кнопки автозаполнения в 1 клик):

• Клиент: client@booking.local / client12345 — запись к мастеру, проверка занятых слотов, отмена визита в ЛК.

• Администратор: admin@booking.local / admin12345 — модерация заявок, изменение расписания смен и проверка мгновенного обновления слотов на главной странице.